# Loan Default Risk Predictor

**Project Documentation**

Tech stack: Python, Pandas, Scikit-learn, Streamlit, Matplotlib

## 1. Abstract

The Loan Default Risk Predictor is a machine learning web application that estimates how likely a loan applicant is to default on a loan. The user enters details such as annual income, loan amount, credit score, credit history, employment history and number of existing loans. Two models, Logistic Regression and Random Forest, predict the probability of default, and the application converts that probability into one of three easy-to-understand categories: **Low**, **Medium** or **High** risk. The application also shows model performance metrics and charts, so the user can see how reliable the predictions are.

## 2. Introduction

Banks and lending institutions lose money when borrowers fail to repay. Traditionally, loan officers judge applications manually, which is slow, inconsistent and prone to personal bias. Machine learning can learn patterns from past loan records and give a fast, data-driven risk estimate for every new applicant.

This project builds a complete, working example of that idea: data preparation, model training, evaluation and an interactive user interface, all in Python.

## 3. Problem Statement

Given an applicant's financial profile, predict whether the applicant is likely to default on the loan, and present the result as a clear risk category along with evidence of how well the model performs.

This is a **binary classification** problem:

- Class 0: the applicant repays the loan (No Default)
- Class 1: the applicant fails to repay (Default)

## 4. Objectives

- Collect applicant details through a simple web form.
- Train and compare two ML models for default prediction.

- Display the default probability and a Low / Medium / High risk category.
- Evaluate the models using precision, recall, F1-score, accuracy, ROC-AUC and the confusion matrix.
- Present all results as charts that non-technical users can understand.

## 5. Usefulness and Applications

| Who | How it helps |
|---|---|
| Banks and NBFCs | Faster first-level screening of loan applications and consistent decisions |
| Loan officers | A second opinion that supports, but does not replace, human judgement |
| Risk managers | Early identification of high-risk applicants and portfolio monitoring |
| Applicants | Understanding which factors (credit score, loan-to-income ratio) affect their risk |
| Students and learners | A complete end-to-end example of a classification project with deployment |

Key benefits:

- **Speed:** a prediction is produced in a fraction of a second.
- **Consistency:** the same inputs always give the same output.
- **Transparency:** feature importance and metrics show why and how well the model works.
- **Lower losses:** catching likely defaulters early reduces bad debt.

## 6. Technology Stack

| Technology | Purpose |
|---|---|
| Python | Core programming language |
| Pandas and NumPy | Data creation, cleaning and manipulation |
| Scikit-learn | Model training, preprocessing and evaluation metrics |

| Technology | Purpose |
|---|---|
| Matplotlib | Charts: bar chart, ROC curve, confusion matrix, feature importance |
| Streamlit | Interactive web interface |

# 7. System Workflow

1. **Data generation or loading:** the dataset is created (or loaded from a CSV file).
2. **Feature engineering:** the loan-to-income ratio is calculated.
3. **Train-test split:** 80% of the data trains the models and 20% tests them.
4. **Model training:** Logistic Regression and Random Forest are trained.
5. **Evaluation:** metrics are calculated on the unseen test data.
6. **Prediction:** the user enters applicant details, and the selected model returns a default probability.
7. **Risk categorisation:** the probability is mapped to Low, Medium or High risk.
8. **Visualisation:** results and charts are shown in the Streamlit app.

# 8. Dataset Information

## 8.1 Source

The project uses a **synthetic (simulated) dataset** generated inside the application using NumPy with a fixed random seed (42), so results are reproducible. The data was designed to follow realistic lending patterns, for example, higher loan-to-income ratios and lower credit scores increase default probability. Because it is simulated, no real customer information is used, which avoids privacy concerns.

The code can be switched to a real dataset (for example, a credit risk dataset from Kaggle) by loading a CSV file with the same column names.

## 8.2 Dataset Summary

| Property | Value |
|---|---|
| Number of records | 5,000 |
| Number of input features | 7 |
| Target variable | default (0 = No Default, 1 = Default) |

| Property | Value |
|---|---|
| Default rate | about 36.9% |
| Training records | 4,000 (80%) |
| Testing records | 1,000 (20%) |
| Split method | Stratified train-test split (random_state = 42) |

## 8.3 Features

| Feature | Description | Type | Range / Distribution |
|---|---|---|---|
| income | Annual income of the applicant (₹) | Numeric | Log-normal, median around ₹6 lakh |
| loan_amount | Loan amount requested (₹) | Numeric | 0.1 to 3 times annual income |
| credit_score | Credit score of the applicant | Numeric | 300 to 900, mean around 680 |
| credit_history_years | Length of credit history in years | Numeric | 0 to 30, mean around 7 |
| employment_years | Years of employment | Numeric | 0 to 35, mean around 6 |
| existing_loans | Number of existing loans | Integer | Poisson, mean around 1.5 |
| loan_to_income | Loan amount divided by annual income (derived) | Numeric | Engineered feature |
| default | Target: 1 if the applicant defaulted | Binary | 0 or 1 |

## 8.4 How the Target Was Generated

Default probability follows a logistic function of the features:

- Higher **loan-to-income ratio** increases risk.
- Higher **credit score** reduces risk.
- Longer **credit history** and longer **employment** reduce risk.
- More **existing loans** increase risk.
- Random noise is added so that the data is not perfectly separable, as in real life.

### 8.5 Data Preprocessing

- Values are clipped to realistic ranges (for example, credit score between 300 and 900).
- The loan-to-income ratio is added as an engineered feature.
- Features are standardised with StandardScaler for Logistic Regression, which is sensitive to feature scale. Random Forest does not need scaling.
- Class imbalance is handled with `class_weight="balanced"` in both models.

## 9. Machine Learning Methodology

### 9.1 Logistic Regression

A linear model that estimates the probability of default using a sigmoid function. It is simple, fast and easy to interpret, which is why it is a standard baseline in credit scoring.

- Preprocessing: StandardScaler inside a Pipeline
- Settings: max_iter = 1000, class_weight = balanced

### 9.2 Random Forest

An ensemble of many decision trees. Each tree votes, and the average vote gives the probability. It captures non-linear relationships and interactions between features, and it provides feature importance scores.

- Settings: n_estimators = 200, max_depth = 8, class_weight = balanced, random_state = 42

### 9.3 Train-Test Split

The data is divided into a training set (80%) and a test set (20%). Models learn only from the training set and are evaluated on the test set, which they have never seen. This gives an honest estimate of real-world performance. Stratification keeps the default ratio the same in both sets.

### 9.4 Risk Categories

| Default probability | Category | Meaning |
|---|---|---|
| Below 30% | Low Risk | Likely to repay; normal approval process |
| 30% to 60% | Medium Risk | Needs additional verification or conditions |
| Above 60% | High Risk | High chance of default; careful review or rejection |

# 10. Evaluation Metrics Explained

| Metric | Meaning | Why it matters here |
|---|---|---|
| Accuracy | Share of all predictions that are correct | Overall correctness, but can mislead when classes are imbalanced |
| Precision | Of applicants predicted to default, how many really defaulted | Low precision means good customers are wrongly rejected |
| Recall | Of applicants who really defaulted, how many were caught | Most important in lending, because a missed defaulter causes direct financial loss |
| F1-score | Harmonic mean of precision and recall | Single balanced score |
| ROC-AUC | Ability to separate defaulters from non-defaulters across all thresholds | 0.5 is random guessing and 1.0 is perfect |
| Confusion matrix | Table of True Negatives, False Positives, False Negatives and True Positives | Shows exactly what kinds of mistakes the model makes |

# 11. Results and Accuracy

Results on the 1,000-record test set:

| Model | Accuracy | Precision | Recall | F1-score | ROC-AUC |
|---|---|---|---|---|---|
| Logistic Regression | 0.736 | 0.613 | 0.770 | 0.683 | 0.815 |
| Random Forest | 0.745 | 0.632 | 0.740 | 0.682 | 0.803 |

## Interpretation

- Both models reach about **74% accuracy** and an F1-score of about **0.68**.
- ROC-AUC above 0.80 shows that both models separate defaulters from non-defaulters well.
- **Logistic Regression has higher recall (0.77)**, so it catches more real defaulters. This is valuable when missing a defaulter is costly.

- **Random Forest has slightly higher precision and accuracy**, so it raises fewer false alarms.
- The two models perform very similarly. This is expected, because the synthetic data was generated from a logistic relationship, which favours Logistic Regression.
- Exact values can vary slightly if the random seed or dataset is changed.

### Visual Outputs in the Application

- Bar chart comparing accuracy, precision, recall and F1 for both models
- ROC curves of both models with AUC values
- Confusion matrix for the selected model
- Feature importance chart from the Random Forest

## 12. Application Features

### Predict tab

- Inputs: annual income, loan amount, existing loans, credit score, credit history and employment years
- Output: default probability, risk category (Low, Medium or High) and a colour-coded risk bar
- A sidebar option to choose between the two models

### Model Performance tab

- Metrics table, comparison chart, ROC curve, confusion matrix and feature importance

### Data tab

- Dataset preview and the overall default rate

## 13. Project Structure

| File | Description |
|---|---|
| app.py | Complete application: data generation, model training, evaluation and the Streamlit interface |
| requirements.txt | List of Python libraries needed |

## 14. Installation and Usage

1. Install Python 3.9 or newer.
2. Open a terminal in the project folder.
3. Install the libraries: `pip install -r requirements.txt`
4. Start the application: `streamlit run app.py`
5. The app opens in the browser at `http://localhost:8501`.
6. Enter applicant details in the **Predict** tab and click **Predict Risk**.
7. Open the **Model Performance** tab to view metrics and charts.

## 15. Limitations

- The dataset is synthetic, so the results show how the method works but are not a guarantee of real-world accuracy.
- Only seven features are used. Real credit scoring also uses repayment history, income stability, collateral and more.
- The risk thresholds (30% and 60%) are fixed and should be tuned with a real lender's risk appetite.
- Models are retrained when the app starts and are not saved to disk.
- The tool is meant to support human decisions, not to replace them.
- Real lending models must be checked for fairness and bias across demographic groups.

## 16. Future Scope

- Train on a real dataset (for example, Kaggle credit risk data) and compare results.
- Add more models such as XGBoost or Gradient Boosting.
- Use hyperparameter tuning with GridSearchCV and cross-validation.
- Handle imbalance with SMOTE or threshold tuning.
- Add explainability with SHAP so each prediction shows which factors drove it.
- Save the trained model with joblib and deploy the app on Streamlit Community Cloud.
- Add user login and a database to store prediction history.

## 17. Conclusion

The Loan Default Risk Predictor shows how machine learning can support lending decisions. Logistic Regression and Random Forest both achieved about 74% accuracy and ROC-AUC above 0.80 on the test data, and the Streamlit interface turns the model output

into simple Low, Medium and High risk categories with clear charts. The project covers the full machine learning cycle, from data preparation to model evaluation to deployment, and it can be extended with real data and more advanced models.

## 18. References


- Scikit-learn documentation: https://scikit-learn.org
- Streamlit documentation: https://docs.streamlit.io
- Pandas documentation: https://pandas.pydata.org
- Matplotlib documentation: https://matplotlib.org